

Django Full Stack Twitter Clone (Chai aur Code)

Comprehensive Notes & Implementation Guide (From [01:13:59])

Source Video: Build a full stack project in Django for beginners (Chai aur Code)

YouTube URL: <http://www.youtube.com/watch?v=opzK3E4Xx6o>

Focus Area: Complete Django CRUD Application, Custom User Authentication, Media Configurations, and Jinja Templates integration.

1. Project Setup & Configurations

Before implementing the views and templates, proper configurations in `settings.py` and `urls.py` are established to handle static assets, media uploads, and app registrations.

Media & Static Root Settings

Configuring where uploaded images and static assets are stored [00:19:59]:

```
import os

# Media files configuration
MEDIA_URL = '/media/'
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')

# Static files configuration
STATIC_URL = '/static/'
STATICFILES_DIRS = [os.path.join(BASE_DIR, 'static')]
```

Project-level URLs (`urls.py`)

Wiring project URLs to include app routes and serve media files during development [02:28:57]:

```
from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('tweet/', include('tweet.urls')),
    path('accounts/', include('django.contrib.auth.urls')),
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

2. Database Models (`models.py`)

The `Tweet` model associates each tweet with a built-in auth user using a foreign key, includes a text field with a 240-character limit, an optional image field, and timestamps [00:44:09]:

```

from django.db import models
from django.contrib.auth.models import User

class Tweet(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    text = models.TextField(max_length=240)
    photo = models.ImageField(upload_to='photos/', blank=True, null=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return f"{self.user.username} - {self.text[:10]}"

```

3. Forms (`forms.py`)

Django ModelForms simplify data handling by automatically generating form fields from models [00:51:29]:

```

from django import forms
from .models import Tweet
from django.contrib.auth.models import User
from django.contrib.auth.forms import UserCreationForm

class TweetForm(forms.ModelForm):
    class Meta:
        model = Tweet
        fields = ['text', 'photo']

class UserRegistrationForm(UserCreationForm):
    email = forms.EmailField(required=True)

    class Meta(UserCreationForm.Meta):
        model = User
        fields = ['username', 'email']

```

4. Application Views (`views.py`)

Views implement the core CRUD functionality, checking HTTP methods, authenticating users, and handling model forms [00:53:21]:

```

from django.shortcuts import render, get_object_or_404, redirect
from .models import Tweet
from .forms import TweetForm, UserRegistrationForm
from django.contrib.auth.decorators import login_required

def index(request):
    return render(request, 'index.html')

def tweet_list(request):
    tweets = Tweet.objects.all().order_by('-created_at')
    return render(request, 'tweet_list.html', {'tweets': tweets})

@login_required
def tweet_create(request):
    if request.method == "POST":
        form = TweetForm(request.POST, request.FILES)
        if form.is_valid():
            tweet = form.save(commit=False)
            tweet.user = request.user
            tweet.save()
            return redirect('tweet_list')
    else:
        form = TweetForm()
    return render(request, 'tweet_form.html', {'form': form})

@login_required
def tweet_edit(request, tweet_id):
    tweet = get_object_or_404(Tweet, pk=tweet_id, user=request.user)
    if request.method == 'POST':
        form = TweetForm(request.POST, request.FILES, instance=tweet)
        if form.is_valid():
            tweet = form.save(commit=False)
            tweet.user = request.user
            tweet.save()
            return redirect('tweet_list')
    else:
        form = TweetForm(instance=tweet)
    return render(request, 'tweet_form.html', {'form': form})

@login_required
def tweet_delete(request, tweet_id):
    tweet = get_object_or_404(Tweet, pk=tweet_id, user=request.user)
    if request.method == 'POST':
        tweet.delete()
        return redirect('tweet_list')
    return render(request, 'tweet_confirm_delete.html', {'tweet': tweet})

def register(request):
    if request.method == 'POST':
        form = UserRegistrationForm(request.POST)
        if form.is_valid():
            user = form.save()
            from django.contrib.auth import login
            login(request, user)
            return redirect('tweet_list')
    else:

```

```
form = UserRegistrationForm()  
return render(request, 'registration/register.html', {'form': form})
```

5. Authentication Settings & Security

To ensure proper routing when users attempt to access protected views (like tweet creation), login redirects are configured in `settings.py` [01:47:37]:

```
LOGIN_URL = '/accounts/login/'  
LOGIN_REDIRECT_URL = '/tweet/'  
LOGOUT_REDIRECT_URL = '/tweet/'
```

Additionally, template-level checks ensure that edit and delete options are rendered only for the tweet's owner:

```
{% if tweet.user == request.user %} [01:57:14].
```